

LAPORAN AKHIR PROJECT BASED LEARNING (PJBL)

Mata Kuliah Algoritma & Struktur Data (TPL2106)

Tahun Akademik 2025/2026

Aplikasi Editor Teks Sederhana (CLI) Berbasis Python

Dosen : Dr. Shelve Nidya Neyman S.Kom., M.Si.

Disusun untuk memenuhi tugas akhir Project-Based Learning (PBL)

Mata Kuliah Algoritma & Struktur Data (TPL2106)

Disusun oleh:



Nama NIM

Muhammad Rafif Fawwaz	J0403251011
Farras Rahman Hakim	J0403251088
Nandana Ahza Fakhrullah	J0403251167

Program Studi Teknologi Rekayasa Perangkat Lunak Sekolah

Vokasi IPB University

2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya, sehingga kami dapat menyelesaikan pembuatan program beserta penyusunan laporan *project* mata kuliah Algoritma dan Struktur Data yang berjudul "**Aplikasi Editor Teks Sederhana (CLI) Berbasis Python**" ini dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas kelompok pada mata kuliah Algoritma dan Struktur Data (TPL2106) di Program Studi Teknologi Rekayasa Perangkat Lunak, Sekolah Vokasi, Institut Pertanian Bogor.

Dalam proses penyusunan laporan dan pengembangan aplikasi ini, kami menyadari bahwa kelancaran dan keberhasilannya tidak terlepas dari bantuan, bimbingan, dan dukungan dari berbagai pihak. Oleh karena itu, pada kesempatan ini kami ingin mengucapkan terima kasih yang sebesar-besarnya kepada:

1. Ibu Dr. Shelve Nidya Neyman S.Kom., M.Si., selaku Dosen Pengampu mata kuliah Algoritma dan Struktur Data yang telah memberikan bekal ilmu, bimbingan, serta arahan yang sangat berharga selama proses perkuliahan.
2. Seluruh anggota kelompok 6 yang telah bekerja keras, membagi waktu, serta berkolaborasi dengan sangat baik dalam merancang logika struktur data (*Linked List*, *Stack*), hingga menyusun kode program dan memecahkan *bug* bersama-sama.
3. Semua pihak yang tidak dapat kami sebutkan satu per satu, yang telah memberikan dukungan baik secara moril maupun materil.

Kami menyadari sepenuhnya bahwa laporan dan program aplikasi ini masih jauh dari kata sempurna. Oleh karena itu, kami sangat terbuka dan mengharapkan segala bentuk kritik serta saran yang membangun dari dosen pengampu maupun pembaca demi perbaikan dan penyempurnaan karya kami di masa mendatang.

Akhir kata, semoga laporan dan *project* ini dapat memberikan manfaat, wawasan, serta referensi yang berguna bagi siapapun yang membacanya.

Contents

LAPORAN AKHIR PROJECT BASED LEARNING (PJBL).....	1
Mata Kuliah Algoritma & Struktur Data (TPL2106).....	1
Tahun Akademik 2025/2026.....	1
KATA PENGANTAR.....	2
BAB I. PENDAHULUAN	4
1.1 Latar Belakang.....	4
1.2 Tujuan Proyek.....	4
1.3 Ruang Lingkup	4
BAB II. ANALISIS DAN PERANCANGAN.....	5
2.1 Deskripsi Sistem	5
2.2 Struktur Data yang Digunakan	6
Alasan Pemilihan Struktur Data.....	7
2.3 Algoritma yang Digunakan	7
Searching.....	7
Sorting.....	7
2.4 Diagram Alir Program (Flowchart)	8
BAB III. IMPLEMENTASI PROGRAM.....	11
3.1 Tampilan Menu Utama	11
3.2 Fitur Create	11
3.3 Fitur Read	12
3.4 Fitur Update	12
3.5 Fitur Delete	13
3.6 Fitur Searching.....	14
3.7 Fitur Sorting.....	14
3.8 Penyimpanan Data (File Handling)	15
3.9 Validasi Input dan Error Handling.....	16
BAB IV. PENGUJIAN DAN ANALISIS	18
4.1 Skenario Pengujian.....	18
4.2 Kelebihan Sistem	19
4.3 Keterbatasan Sistem	19
BAB V. KENDALA DAN SOLUSI	20
BAB VI. PEMBAGIAN TUGAS ANGGOTA	21
BAB VII. KESIMPULAN DAN SARAN	22
Lampiran E. Source Code Utama	27

BAB I. PENDAHULUAN

1.1 Latar Belakang

Proyek ini dibuat untuk menyediakan solusi pengolahan teks berbasis terminal yang ringan namun mampu melacak riwayat perubahan secara efisien. Aplikasi ini membantu pengelolaan dokumen teks dengan mengimplementasikan struktur data *Linked List* untuk menyimpan koleksi dokumen secara dinamis. Selain itu, program ini menerapkan struktur data *Stack* untuk mengelola fitur Undo dan Redo, memberikan fleksibilitas saat mengedit teks layaknya aplikasi editor teks modern. Semua data akan disimpan secara otomatis ke dalam file CSV agar informasi tersimpan permanen dan tidak hilang ketika aplikasi ditutup.

1.2 Tujuan Proyek

1. Membuat aplikasi editor teks interaktif berbasis terminal (CLI) menggunakan bahasa Python.
2. Mengimplementasikan dan memadukan struktur data *Linked List* (sebagai penyimpanan koleksi utama) dan *Stack* (sebagai penyimpanan riwayat aksi).
3. Menerapkan algoritma pencarian (Linear Search) dan pengurutan (Bubble, Selection, dan Insertion Sort) dalam kasus pengelolaan teks nyata.
4. Menerapkan konsep *file handling* (CSV/TXT) untuk mendukung penyimpanan data secara persisten.

1.3 Ruang Lingkup

- Platform aplikasi menggunakan *Command Line Interface* (Console).
- Ditulis sepenuhnya dalam bahasa Python.
- Menggunakan format penyimpanan file .csv untuk *database* internal dan fitur ekspor ke .txt.
- Memiliki fitur utama: CRUD Dokumen, Undo/Redo, *Searching*, dan *Sorting*.

BAB II. ANALISIS DAN PERANCANGAN

2.1 Deskripsi Sistem

Aplikasi Editor Teks Sederhana ini merupakan program pengolah kata berbasis terminal atau *Command Line Interface* (CLI) yang dikembangkan menggunakan bahasa pemrograman Python. Dirancang dengan arsitektur modular, aplikasi ini memisahkan setiap fungsi ke dalam modul tersendiri agar struktur kode lebih rapi dan mudah dirawat. Pada menu utama, sistem menggunakan layout tiga kolom yang dipetakan secara efisien lewat konsep *dictionary mapping* guna menghindari baris kode if-elif yang terlalu panjang.

Secara garis besar, fungsionalitas dan operasional utama sistem ini dibagi menjadi empat bagian:

1. Manajemen Dokumen Teks (CRUD)

- **Struktur Objek Berkas:** Setiap berkas teks direpresentasikan oleh objek dari kelas *Document*. Objek ini menyimpan informasi penting (metadata) dokumen, seperti *doc_id* (*auto-increment*), *title* (judul), *lines* (list untuk menampung baris teks), serta catatan waktu pembuatan (*created_at*) dan perubahan (*updated_at*).
- **Penyimpanan Koleksi Utama:** Untuk menampung semua dokumen di memori, sistem menggunakan struktur data *Singly Linked List* kustom. Saat dokumen baru dibuat, objek tersebut akan ditambahkan sebagai simpul terakhir (*end-node*) setelah sistem menelusuri pointer dari simpul awal (*head*) hingga ujung rantai.
- **Mode Editor Interaktif:** Ketika suatu dokumen dibuka, pengguna akan masuk ke "Mode Editor" yang berjalan dalam sub-loop khusus. Di sini, pengguna bisa memanipulasi teks secara spesifik per baris, mulai dari menambah baris baru secara terus-menerus (ketik selesai untuk berhenti), menyunting isi baris tertentu, hingga menghapus baris yang dipilih.

2. Mekanisme Fungsi Undo & Redo (Dual-Stack)

- **Perekaman Riwayat Berbasis Tuple:** Setiap ada perubahan data—seperti membuat dokumen baru, mengedit kalimat, atau menghapus baris—*DocumentManager* akan langsung membungkus detail aksi tersebut ke dalam bentuk *Tuple* (berisi jenis operasi, indeks, teks lama, dan teks baru). *Tuple* ini kemudian dimasukkan (*push*) ke dalam *undo_stack*.
- **Pembalikan Aksi via Prinsip LIFO:** Saat pengguna memilih menu *Undo*, sistem akan

menarik (*pop*) aksi terakhir dari puncak tumpukan berdasarkan prinsip LIFO (*Last In, First Out*) untuk membalikkan efeknya pada *Linked List* utama. Aksi yang dibatalkan ini lalu dipindahkan ke *redo_stack* agar bisa diulangi kembali via menu *Redo*.

- **Aturan Modifikasi Baru:** Sama seperti aplikasi pengolah kata modern, jika pengguna melakukan perubahan data baru setelah menekan *Undo*, maka seluruh riwayat aksi yang tersimpan di dalam *Redo Stack* otomatis akan dihapus bersih (`redo_stack.clear()`).

3. Fitur Pencarian dan Pengurutan (Searching & Sorting)

- **Pencarian Linear:** Sistem menggunakan algoritma *Linear Search* yang bersifat *case-insensitive* (tidak peka huruf besar/kecil) untuk mencari dokumen berdasarkan judul atau kata di dalam isi teks. Jika mencari berdasarkan isi, sistem akan menampilkan nama dokumen beserta nomor baris spesifik tempat kata kunci tersebut ditemukan.
- **Pengurutan Fleksibel:** Pengguna bisa mengurutkan daftar dokumen berdasarkan tiga kriteria: alfabet Judul lewat algoritma **Bubble Sort** (dengan optimasi bendera penanda agar berhenti lebih awal jika sudah rapi), waktu pembuatan dokumen lewat **Selection Sort**, dan jumlah baris teks lewat **Insertion Sort**. Proses pengurutan ini dilakukan pada salinan data berkas temporer, sehingga urutan fisik pointer asli pada *Linked List* utama tetap aman dan tidak berubah.

4. Persistensi Data dan Sinkronisasi Berkas

- **Serialisasi Berkas CSV:** Agar dokumen tidak hilang saat program ditutup, sistem menggunakan teknik *file handling* untuk menyimpan data secara permanen ke file `documents.csv`. Karena file CSV biasa sulit menyimpan teks banyak baris (*multiline*) dalam satu kolom, sistem menggabungkan semua baris tersebut menjadi satu string utuh dengan pemisah unik `|||` sebelum ditulis ke file. Saat program dibuka kembali, string ini akan dipecah lagi (`.split("|||")`) menjadi bentuk list semula.
- **Ekspor TXT & Auto-Save:** Selain database CSV, sistem menyediakan fitur untuk mengeksport isi dokumen pilihan menjadi berkas teks biasa berekstensi `.txt`. Fungsi *auto-save* ke berkas CSV juga akan langsung berjalan secara otomatis tepat setelah pengguna selesai melakukan modifikasi data apa pun, sehingga progres mengetik selalu aman dari risiko kehilangan data.

2.2 Struktur Data yang Digunakan

Singly Linked List: Berfungsi untuk menyimpan koleksi seluruh dokumen teks di dalam memori saat program berjalan.

Stack: Terdapat dua buah Stack, yaitu `undo_stack` untuk menyimpan aksi pengguna sebelumnya, dan `redo_stack` untuk menyimpan aksi yang baru saja di-undo.

Alasan Pemilihan Struktur Data

Linked List dipilih karena mendukung penyisipan dan penghapusan *node* dokumen di posisi tertentu secara lebih efisien dibandingkan *Array/List* biasa, cukup dengan memodifikasi *pointer* antar *node*. Hal ini krusial saat fitur Undo dijalankan untuk mengembalikan dokumen yang tak sengaja terhapus ke posisi indeks aslinya.

Stack dipilih karena mengikuti prinsip LIFO (*Last In, First Out*). Prinsip ini identik dengan cara kerja *Undo*, di mana perubahan teks yang dilakukan paling akhir adalah hal pertama yang harus dibatalkan/dihapus ketika *Undo* dipanggil.

2.3 Algoritma yang Digunakan

Searching

- Linear Search

Alasan penggunaan:

Menggunakan **Linear Search** untuk mencari kata kunci secara berurutan, baik berdasarkan pencarian judul dokumen maupun konten (teks di dalam baris). Penelusuran satu per satu dipilih karena pengaksesan elemen dalam *Linked List* paling cocok dilakukan secara *linear traversal*.

Sorting

- Bubble Sort

Alasan penggunaan:

Digunakan untuk mengurutkan daftar dokumen berdasarkan Judul (A-Z/Z-A).

- Selection Sort

Alasan penggunaan:

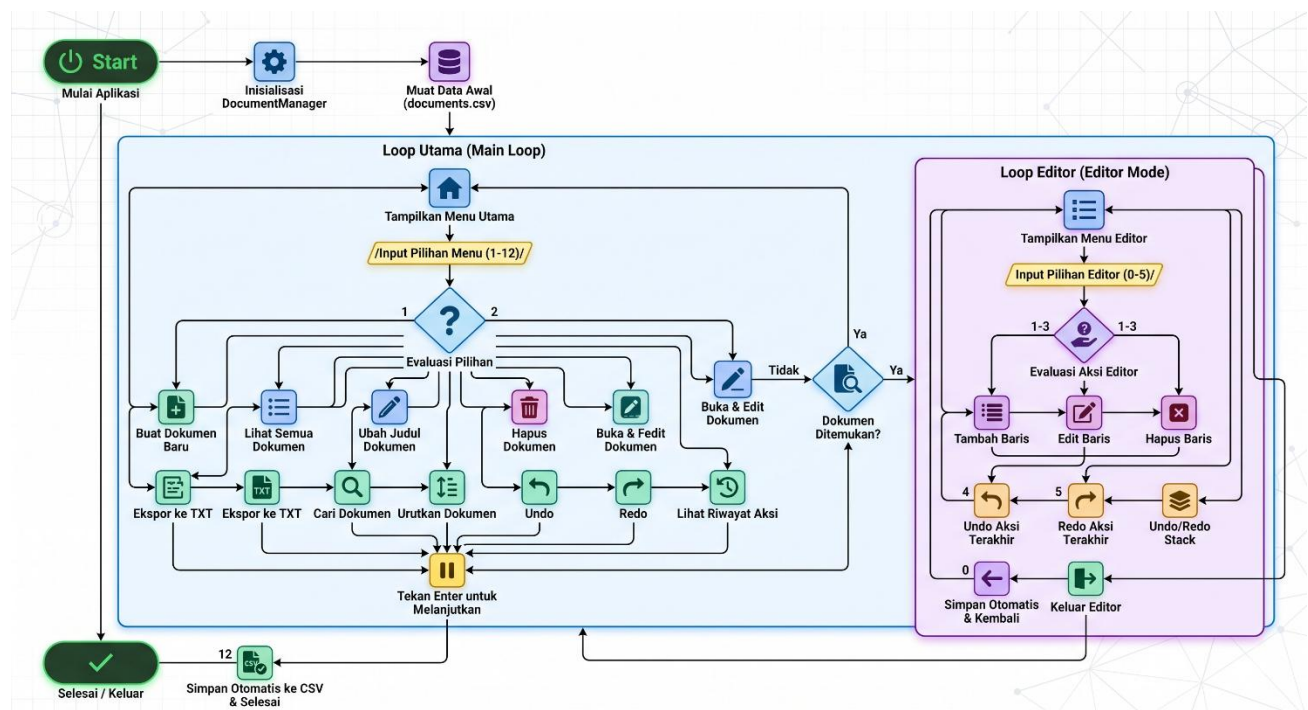
Digunakan untuk mengurutkan daftar dokumen berdasarkan urutan waktu (Terbaru/Terlama).

- Insertion Sort

Alasan penggunaan:

Digunakan untuk mengurutkan berdasarkan Jumlah Baris teks. Algoritma ini diimplementasikan untuk mendemonstrasikan perbandingan teknik komputasi konvensional saat mengurutkan properti *Linked List* yang disalin.

2.4 Diagram Alir Program (Flowchart)



2.5 Struktur Program

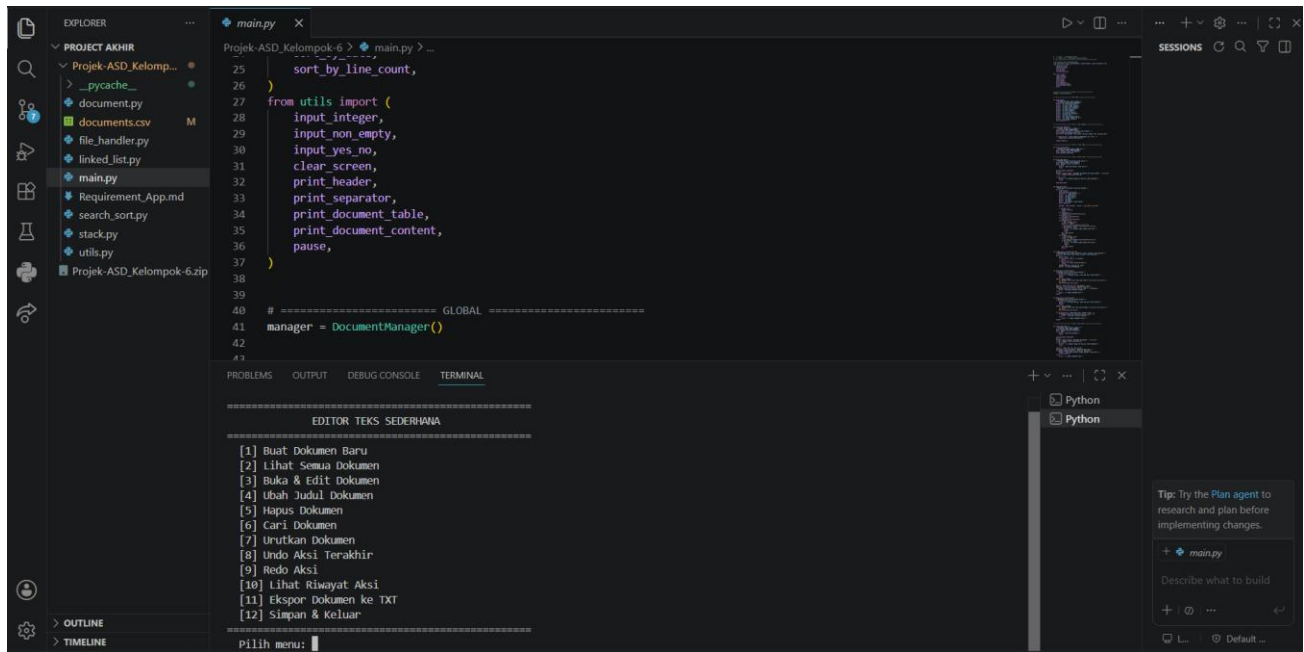
Untuk menjaga kode program tetap rapi, mudah dipahami, dan gampang dirawat, seluruh kode aplikasi ini dibagi ke dalam beberapa modul file berikut:

File	Deskripsi
main.py	Berisi titik masuk program, inisialisasi menu, dan interaksi CLI (I/O).
document.py	Berisi definisi dari kelas Document dan DocumentManager. Di sinilah logika utama pemrosesan dokumen (CRUD) berada, termasuk fungsi perekaman riwayat perubahan teks untuk mekanisme <i>Undo</i> dan <i>Redo</i> .
linked_list.py	Menyimpan implementasi kustom dari struktur data <i>Singly Linked List</i> , lengkap dengan pembuatan komponen <i>Node</i> . Modul ini menangani operasi penyimpanan koleksi berkas secara dinamis di dalam memori.
stack.py	Menyimpan implementasi kustom dari struktur data <i>Stack</i> dengan memanfaatkan <i>list</i> bawaan Python. Struktur ini bekerja menggunakan prinsip LIFO (<i>Last In, First Out</i>) khusus untuk mengelola riwayat aksi pengguna.
search_sort.py	Menampung modularisasi dari seluruh logika pengolahan data. Di dalamnya terdapat fungsi pencarian menggunakan <i>Linear Search</i> , serta tiga algoritma pengurutan dokumen yang berbeda, yaitu <i>Bubble Sort</i> , <i>Selection Sort</i> , dan <i>Insertion Sort</i> .
file_handler.py	Bertanggung jawab penuh atas manajemen berkas eksternal, mulai dari proses memuat data lama, memperbarui isi database secara otomatis (<i>auto-save</i>), hingga fungsi ekspor dokumen pilihan ke dalam format berkas plain teks (.txt).

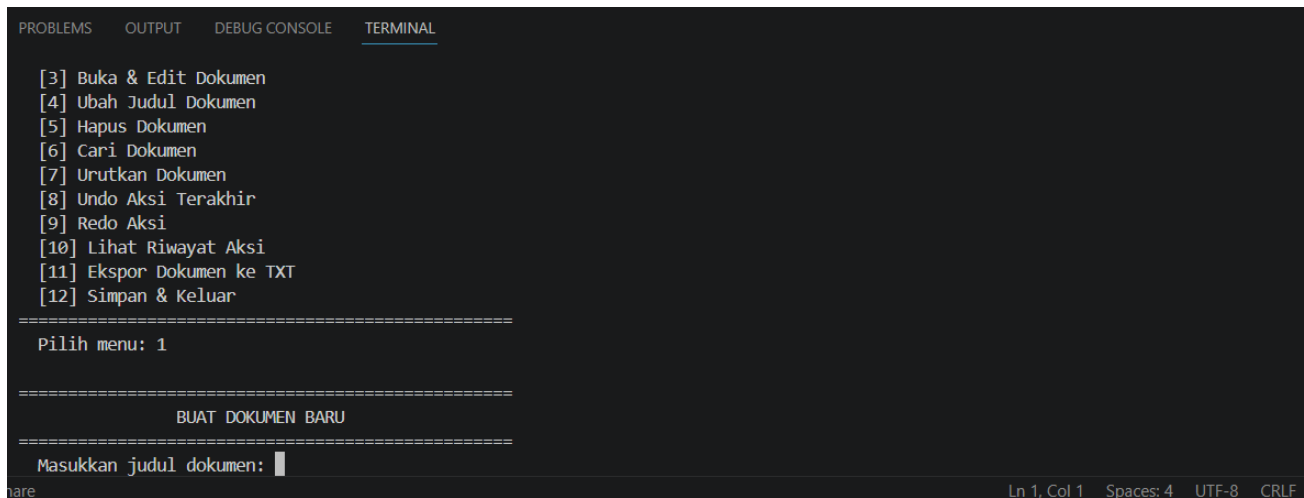
utils.py	Berisi fungsi-fungsi utilitas atau pembantu, seperti fungsi pembersihan layar terminal, pencetakan elemen tabel agar tampilan visual CLI terlihat rapi, hingga fungsi validasi masukan agar program tidak mengalami <i>crash</i> saat pengguna salah mengetik.
document.csv	Berkas penyimpanan permanen datar (<i>flat-file</i>) yang bertindak sebagai database internal aplikasi. Berkas ini menyimpan data fisik dokumen secara terstruktur ke dalam beberapa kolom data, yaitu doc_id, title, lines (di mana baris-baris teks digabungkan menjadi satu string tunggal menggunakan pembatas khusus), created_at, serta updated_at. Berkas ini akan otomatis dibaca saat aplikasi pertama kali dijalankan dan diperbarui setiap kali ada perubahan data yang sukses dilakukan.

BAB III. IMPLEMENTASI PROGRAM

3.1 Tampilan Menu Utama



3.2 Fitur Create



Sistem meminta pengguna memasukkan judul. Objek Document baru akan diinisialisasi secara otomatis dengan ID, tanggal, lalu disambungkan (menggunakan method `.append()`) ke posisi paling ujung dari struktur *Linked List*.

3.3 Fitur Read

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

[9] Redo Aksi
[10] Lihat Riwayat Aksi
[11] Ekspor Dokumen ke TXT
[12] Simpan & Keluar
=====
Pilih menu: 2

=====
                DAFTAR SEMUA DOKUMEN
=====
  ID    Judul                Baris    Dibuat
-----
  1     Tugas tekmul         4      2026-02-25 12:11:33
  2     Plottingan          0      2026-05-19 16:07:23
  3     12                   0      2026-06-26 14:31:38

Tekan Enter untuk melanjutkan...
```

Program menampilkan tabel seluruh dokumen dengan menelusuri (melakukan *traversal*) elemen *Linked List* secara keseluruhan mulai dari *Head* hingga elemen menunjuk ke *None*.

3.4 Fitur Update

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

[9] Redo Aksi
[10] Lihat Riwayat Aksi
[11] Ekspor Dokumen ke TXT
[12] Simpan & Keluar
=====
Pilih menu: 3

=====
                BUKA & EDIT DOKUMEN
=====
  ID    Judul                Baris    Dibuat
-----
  1     Tugas tekmul         4      2026-02-25 12:11:33
  2     Plottingan          0      2026-05-19 16:07:23
  3     12                   0      2026-06-26 14:31:38

Masukkan ID dokumen yang ingin dibuka: 
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

Jml Baris: 4
-----
 1 | Assalamualaikum warahmatullahi wabarakatu
 2 | 0
 3 | 0
 4 | 0
-----

--- Menu Editor ---
[1] Tambah Baris
[2] Edit Baris
[3] Hapus Baris
[4] Undo
[5] Redo
[0] Kembali ke Menu Utama
-----

Pilih: █
```

Terdapat fitur untuk mengubah judul dan "Mode Editor" di dalam dokumen. Mode editor membuka menu *sub-loop* tempat pengguna dapat menunjuk nomor baris spesifik, lalu meng- *update* (edit/tambah/hapus) data teks di sana.

3.5 Fitur Delete

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

[9] Redo Aksi
[10] Lihat Riwayat Aksi
[11] Ekspor Dokumen ke TXT
[12] Simpan & Keluar
=====
Pilih menu: 5
=====
HAPUS DOKUMEN
=====
ID      Judul      Baris  Dibuat
-----
1      Tugas tekmul      4      2026-02-25 12:11:33
2      Plottingan        0      2026-05-19 16:07:23
3      12                0      2026-06-26 14:31:38

Masukkan ID dokumen yang ingin dihapus: █
```

Mengambil lokasi posisi *node* di *Linked List* dari dokumen terkait. Data dokumen tersebut terlebih dahulu dipaketkan menjadi *Dictionary* dan ditaruh di *undo_stack* sebagai pencadangan. Lalu, koneksi penunjuk *next* pada simpul sebelumnya ditautkan ke simpul sesudahnya agar simpul terhapus (metode *bypass*).

3.6 Fitur Searching

Pada fitur pencarian, sistem mengimplementasikan algoritma **Linear Search** (pencarian sekuensial). Cara kerjanya adalah dengan menelusuri seluruh koleksi dokumen satu per satu dari awal sampai akhir tanpa ada dokumen yang terlewat. Melalui modul `search_sort.py`, pencarian ini dibuat menjadi dua mode dinamis yang semuanya bersifat *case-insensitive* (mengabaikan perbedaan huruf besar dan kecil):

- **Pencarian Berdasarkan Judul (`search_by_title`):** Sistem akan melakukan iterasi pada daftar dokumen, mengubah judul dokumen dan kata kunci menjadi huruf kecil lewat fungsi `.lower()`, lalu menyaring dokumen yang judulnya mengandung kata kunci tersebut.
- **Pencarian Berdasarkan Konten (`search_by_content`):** Prosesnya bekerja lebih dalam, di mana sistem melakukan kalang bersarang (*nested loop*). Selain memeriksa tiap dokumen, sistem memanfaatkan fungsi `enumerate()` untuk membedah setiap baris teks di dalam dokumen tersebut. Begitu kata kunci ditemukan di suatu kalimat, indeks posisinya akan disimpan ke dalam sebuah list baris yang cocok, sehingga sistem bisa menampilkan rincian nomor barisnya secara akurat kepada pengguna.

3.7 Fitur Sorting

Untuk fitur pengurutan, sistem menyediakan menu yang sangat fleksibel dengan menerapkan tiga jenis algoritma pengurutan yang berbeda. Satu catatan penting, proses *sorting* ini beroperasi pada salinan list data temporer (`list(documents_list)`). Langkah ini sengaja diambil agar proses pengurutan data visual hanya bersifat sementara dan tidak merusak atau mengacak urutan fisik penunjuk pointer asli pada *Singly Linked List* utama.

Berikut adalah penjelasan lengkap dari ketiga algoritma pengurutan yang digunakan di dalam sistem:

- **Bubble Sort (Digunakan untuk Mengurutkan Judul Dokumen)** Algoritma ini bekerja dengan membandingkan dua elemen data yang posisinya bersebelahan secara berulang-ulang. Jika urutan alfabet judul dokumen pertama lebih besar dari dokumen kedua (untuk pilihan *ascending* A-Z), posisinya akan langsung ditukar. Dokumen dengan nilai terbesar akan pelan-pelan "menggelembung" naik ke posisi akhir. Untuk menghemat waktu komputasi, fungsi `sort_by_title` ini sudah dilengkapi dengan optimasi bendera (*swapped flag*); jika dalam satu putaran penuh tidak ada satu pun posisi data yang ditukar, algoritma akan berhenti lebih awal karena data dianggap sudah rapi.
- **Selection Sort (Digunakan untuk Mengurutkan Tanggal Dibuat/Diubah)** Cara kerja

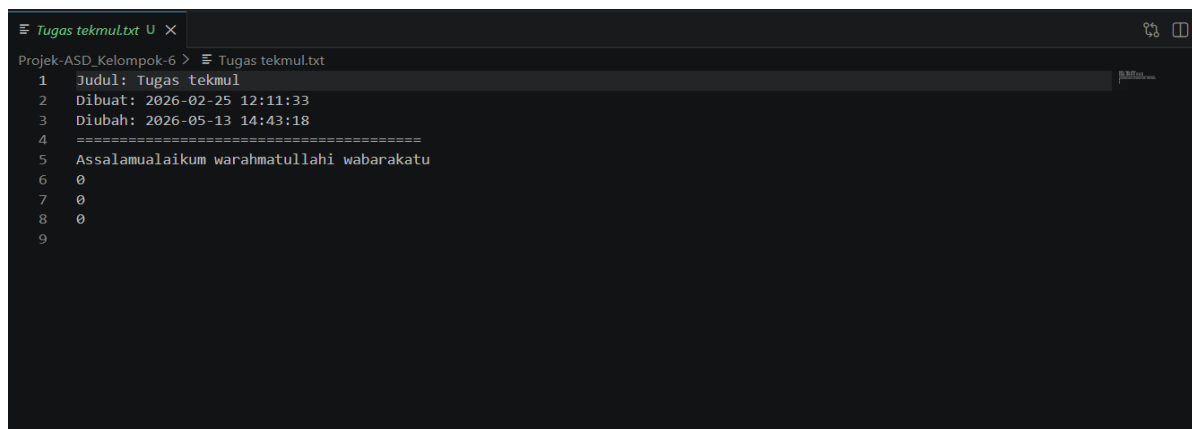
algoritma ini mirip dengan cara manusia memilih kartu terkecil atau terbesar dari tangan secara manual. Sistem akan memisahkan list menjadi dua bagian: bagian yang sudah urut dan bagian yang belum urut. Pada setiap putaran, sistem memindai area yang belum urut untuk mencari nilai stempel waktu (*timestamp*) terkini atau terlama, menyimpan indeks targetnya (*target_idx*), lalu menukarnya ke posisi paling depan dari area yang belum urut tersebut. Keunggulan dari metode ini adalah jumlah pertukaran data (*swap*) menjadi sangat minimal, karena pertukaran hanya dilakukan satu kali di akhir setiap putaran.

- **Insertion Sort (Digunakan untuk Mengurutkan Berdasarkan Jumlah Baris Teks)**

Algoritma ini bekerja dengan cara menyisipkan sebuah elemen data langsung ke posisi yang tepat di antara elemen-elemen lain yang sudah terurut sebelumnya. Sistem mengambil dokumen satu per satu mulai dari indeks kedua, lalu menjadikannya sebagai kunci penentu (*key*). Jumlah baris dari dokumen kunci ini akan dibandingkan dengan dokumen di sebelah kirinya. Selama dokumen di sebelah kiri memiliki jumlah baris yang lebih banyak, posisinya akan digeser ke kanan untuk membuka ruang, lalu dokumen kunci tadi akan langsung disisipkan di celah kosong yang tepat.

3.8 Penyimpanan Data (File Handling)

- **Format file:** Menggunakan format documents.csv untuk memori komputasi internal, dan .txt untuk ekspor teks output per satuan.
- **Cara membaca file:** Menggunakan modul bawaan `csv.reader()`. Program membaca baris `row[2]` dan memecahnya kembali menjadi struktur kalimat list menggunakan `.split("|||")`.
- **Cara menyimpan file:** Data diekstrak dan digabung satu sama lain (semua baris dikonversi menjadi satu sel string tunggal dengan pemisah `|||`). Fungsi `writer` menumpuk keseluruhan isi CSV secara `overwrite` dengan nilai Linked List terbaru. Hal ini dilakukan otomatis oleh sistem saat data berubah.



```
Tugas tekmul.txt
Projek-ASD_Kelompok-6 > Tugas tekmul.txt
1  Judul: Tugas tekmul
2  Dibuat: 2026-02-25 12:11:33
3  Diubah: 2026-05-13 14:43:18
4  =====
5  Assalamualaikum warahmatullahi wabarakatu
6  0
7  0
8  0
9
```

3.9 Validasi Input dan Error Handling

Untuk memastikan aplikasi berjalan stabil dan tidak berhenti mendadak (crash) akibat kelalaian pengguna, sistem dilengkapi dengan validasi input berlapis yang dikelola melalui modul utilitas dan file handler. Berikut adalah tabel rincian penanganan kesalahan (error handling) yang diterapkan di dalam sistem:

Kesalahan / Kondisi Error	Penanganan oleh Sistem
Memasukkan huruf pada input yang mewajibkan angka (contoh: ID Dokumen, Nomor Baris, Pilihan Menu)	Program menggunakan blok try-except untuk menangkap ValueError. Sistem akan menampilkan pesan <i>"Input harus berupa angka. Coba lagi."</i> dan terus menahan loop hingga pengguna memasukkan angka yang valid.
Input teks dibiarkan kosong atau hanya berisi spasi (contoh: saat mengetik judul dokumen baru)	Fungsi input_non_empty() akan membersihkan input dengan metode .strip(). Jika hasilnya kosong, program menolak input tersebut dengan pesan peringatan dan meminta pengguna mengetik ulang.
Memasukkan angka di luar rentang opsi yang ada (contoh: memilih menu 15 padahal opsi hanya sampai 12)	Fungsi input divalidasi dengan parameter batas bawah (min_val) dan batas atas (max_val). Sistem akan memblokir angka di luar batas, menampilkan peringatan limit nilai, lalu meminta <i>input</i> ulang.
Data yang dicari tidak ditemukan (contoh: memanggil ID yang sudah dihapus atau tidak ada)	Program akan melakukan pengecekan validitas pencarian. Jika kembalian bernilai None, sistem akan menampilkan pesan peringatan ramah (<i>"Dokumen dengan ID X tidak ditemukan"</i>) tanpa menghentikan paksa aplikasi, lalu mengembalikan pengguna ke tampilan layar sebelumnya dengan aman.
File CSV atau TXT belum tersedia (contoh: saat aplikasi pertama kali diunduh dan dihidupkan)	Fungsi pembacaan load_documents() di <i>file handler</i> akan mengecek ketersediaan jalur <i>path</i> menggunakan os.path.exists(). Jika file

	documents.csv tidak ditemukan, program memunculkan notifikasi <i>"File data belum ada. Memulai dengan data kosong"</i> dan menginisialisasi <i>Linked List</i> yang bersih. File baru kemudian akan otomatis diciptakan saat fitur <i>auto-save</i> terpicu.
--	--

BAB IV. PENGUJIAN DAN ANALISIS

4.1 Skenario Pengujian

No	Skenario	Hasil
1	Tambah Data / Isi Dokumen	Saat pembuatan dokumen, program menanya otomatis perihal <i>multiline input</i> . Hasil: Berhasil masuk ke node akhir <i>Linked List</i> .
2	Edit dan Hapus Data Baris	Proses penargetan di indeks tertentu, diganti. Hasil: Berhasil terganti dan berhasil disimpan ke histori Stack.
3	Hapus Dokumen	Mengkonfirmasi ID file yang akan dilompati (<i>bypass</i>) di <i>Linked List</i> . Hasil: Sukses dan ID terhapus.
4	Sorting	Data berjumlah >3 disisipkan lalu diperintah sortir secara "Z-A". Hasil: Bubble sort berhasil menampilkan urutan membalik abjad tanpa kendala.
5	Searching	Kata "Coba" diketik menggunakan karakter kecil "coba". Hasil: <i>Linear Search</i> berhasil menemukannya karena mengadopsi pencarian <i>case-insensitive</i> .
6	Undo & Redo Aksi	Aksi modifikasi (tambah baris/hapus file) dibatalkan via instruksi <i>Undo</i> dan diulangi via instruksi <i>Redo</i> berturut-turut. Hasil: Eksekusi <i>pop()</i> dan metode pembalikan fungsi berbalik berjalan dengan lancar

		merekonstruksi teks.
--	--	----------------------

4.2 Kelebihan Sistem

1. Integritas data terjamin dan disimpan permanen berkat implementasi fitur auto-save setelah action berhasil diselesaikan.
2. Aplikasi tidak rentan mengalami crash mendadak di tengah jalan apabila user secara random salah menekan papan ketik karena memiliki modul `utils.py` yang kuat.
3. Kehadiran fitur Redo yang interaktif di mana stack Redo akan dihapus apabila pengguna mereset rantai masa depan dengan melakukan tindakan baru setelah Undo, identik secara logika seperti program besar Microsoft Word.

4.3 Keterbatasan Sistem

- Semua pengoperasian visual murni masih bergantung pada Terminal CLI statis.
- Kecepatan operasi pengurutan memakan waktu $O(n^2)$ karena tidak menggunakan algoritma logaritmik seperti Quick/Merge Sort.
- Penyimpanan data komputasional yang masih berpusat kepada manipulasi File CSV (*flat file*), belum mengimplementasikan mesin Database Relasional (contoh: PostgreSQL/MySQL).

BAB V. KENDALA DAN SOLUSI

Kendala	Solusi
Satu dokumen bisa memiliki berbagai deretan baris teks (multiline), sementara arsitektur file CSV standar hanya mendukung kompresi satu nilai <i>cell</i> untuk satu kolom. Hal ini membuat kompresi tabel menjadi kacau.	Mengakali celah ini dengan menggabungkan semua kumpulan array teks menjadi serangkaian String tunggal yang direkatkan melalui karakter <i>separator</i> khusus pada waktu penulisan/penyimpanan di <i>file_handler.py</i> , sehingga muat di dalam struktur satu sel CSV.
Jika program dibiarkan mengeksekusi konversi tipe data angka yang diketikkan huruf oleh <i>user</i> (misalnya di ID pencarian), aplikasi akan mengalami <i>crash</i> mendadak (Interrupted Exception).	Membungkus setiap fungsi yang meminta angka dengan skema kalang While Loop yang mengandung mitigasi try-except ValueError, lalu memaksa pengguna mengisi ulang input.

BAB VI. PEMBAGIAN TUGAS ANGGOTA

Nama	Tugas
Muhammad Rafif Fawwaz	(Lead Programmer) Mengimplementasikan logika inti sistem, termasuk operasi CRUD (Create, Read, Update, Delete) dokumen, arsitektur <i>Linked List</i> dan <i>Stack</i> , serta melakukan modularisasi kode untuk memastikan struktur program rapi dan mudah dipelihara.
Farras Rahman Hakim	(Co-Programmer & Code Optimizer) Fokus pada pengembangan modul utilitas (<i>utils.py</i>), implementasi search & sort, melakukan <i>refactoring</i> untuk meningkatkan efisiensi pembacaan kode, serta menangani validasi <i>input</i> sistem untuk meminimalisasi <i>error</i> .
Nandana Ahza Fakrullah	(Project Manager & System Analyst) Bertugas mengelola jalannya proyek dan merancang alur kerja aplikasi (<i>flowchart</i>) dari awal hingga akhir. Selain itu, turut berkontribusi dalam penulisan kode program untuk membantu penyempurnaan fitur, bertanggung jawab sebagai <i>tester</i> utama untuk melakukan pengujian aplikasi secara komprehensif guna memastikan tidak ada <i>bug</i> atau kecacatan sistem sebelum program diselesaikan.

Pembagian tugas di atas merupakan fokus utama masing-masing anggota; namun demikian, seluruh anggota kelompok turut terlibat dalam diskusi perancangan, debugging, serta pengujian aplikasi secara menyeluruh. Kontribusi masing-masing anggota turut tercermin pada riwayat commit (commit history) repositori GitHub kelompok sebagaimana tercantum pada Lampiran D.

BAB VII. KESIMPULAN DAN SARAN

Kesimpulan

- Program "Editor Teks Sederhana" berhasil dijalankan dengan memenuhi semua syarat implementasi CRUD beserta penggunaan kombinasi dua buah instrumen struktur data secara harmonis (Singly Linked List dan Stack).
- Mekanisme prinsip algoritma (seperti Search dan Sort) yang dipelajari selama perkuliahan sukses diimplementasikan dan digunakan secara langsung untuk memanipulasi pengelolaan dokumen dunia nyata secara permanen melalui teknik kompresi sel tabel CSV.

Saran Pengembangan

- Aplikasi selanjutnya dapat ditingkatkan menjadi program berbasis GUI (Grafis) interaktif dengan pustaka semacam Tkinter atau PyQt agar user bisa menggunakan kursor (mouse) dan visual menu Drop-down.
- Fitur penyimpanan internal di kemudian waktu bisa dialihkan menggunakan sistem Query Database formal (DBMS) seperti SQLite agar pemanggilan Search/Sort lebih terindeks dan cepat.

DAFTAR PUSTAKA

ashishps1. (n.d.). Awesome leetcode resources [Source code]. GitHub.

<https://github.com/ashishps1/awesome-leetcode-resources>

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.

GeeksforGeeks. (n.d.). Python data structures and algorithms.

<https://www.geeksforgeeks.org/dsa/python-data-structures-and-algorithms/>

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). Data structures and algorithms in Python. Wiley.

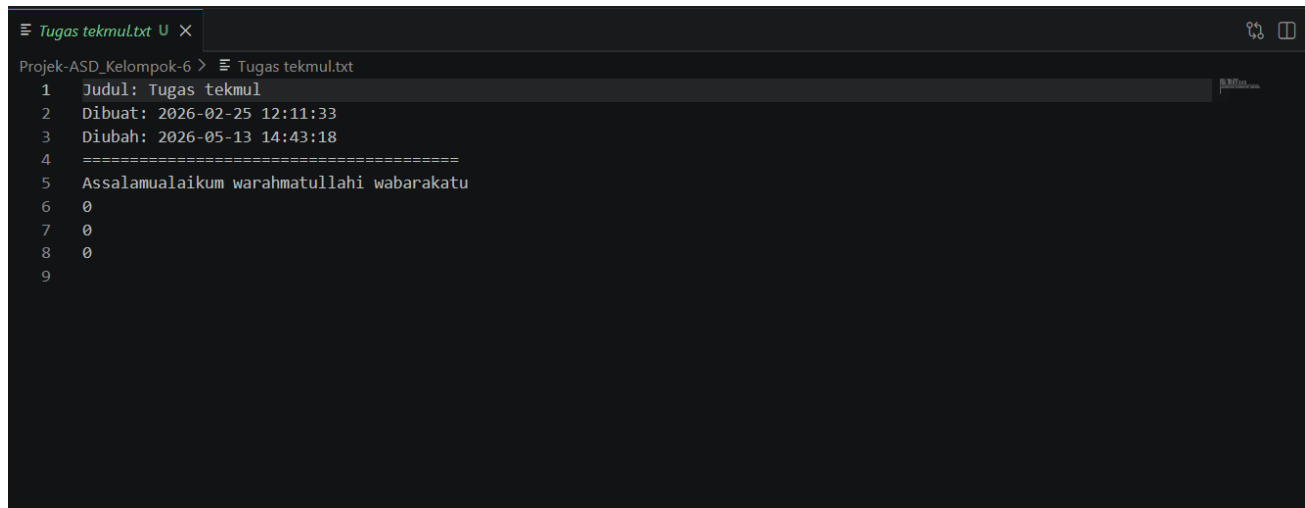
Miller, B., & Ranum, D. (2011). Problem solving with algorithms and data structures using Python. Franklin, Beedle & Associates.

LAMPIRAN

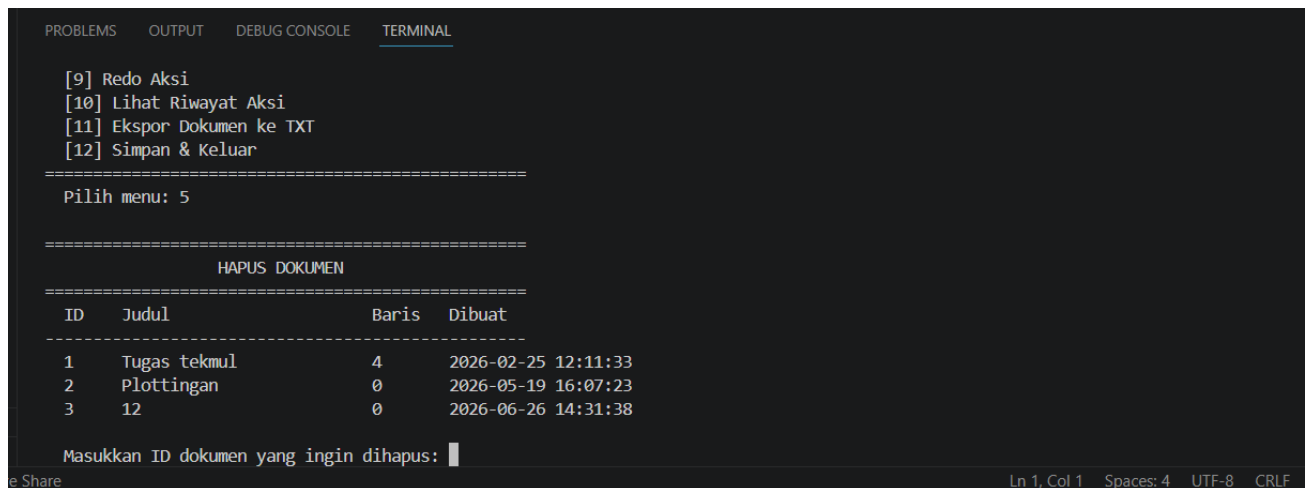
Lampiran A. Link GitHub

https://github.com/RAFIFFZIPB/ASD_B_6

Lampiran B. Screenshot Program



```
Tugas tekmul.txt
Projek-ASD_Kelompok-6 > Tugas tekmul.txt
1  Judul: Tugas tekmul
2  Dibuat: 2026-02-25 12:11:33
3  Diubah: 2026-05-13 14:43:18
4  =====
5  Assalamualaikum warahmatullahi wabarakatu
6   
7   
8   
9
```



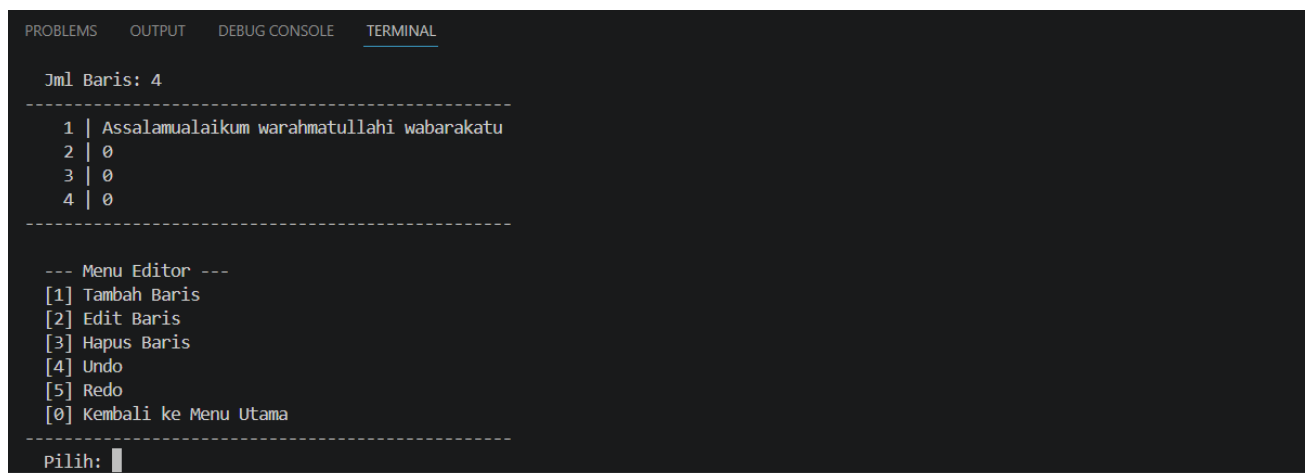
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

[9] Redo Aksi
[10] Lihat Riwayat Aksi
[11] Ekspor Dokumen ke TXT
[12] Simpan & Keluar

=====
Pilih menu: 5

=====
HAPUS DOKUMEN
=====
=====
ID      Judul          Baris  Dibuat
-----
1      Tugas tekmul      4      2026-02-25 12:11:33
2      Plottingan               2026-05-19 16:07:23
3      12                       2026-06-26 14:31:38

Masukkan ID dokumen yang ingin dihapus: █
```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

Jml Baris: 4
-----
1 | Assalamualaikum warahmatullahi wabarakatu
2 |  
3 |  
4 |  
-----

--- Menu Editor ---
[1] Tambah Baris
[2] Edit Baris
[3] Hapus Baris
[4] Undo
[5] Redo
[ ] Kembali ke Menu Utama
-----

Pilih: █
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

[9] Redo Aksi
[10] Lihat Riwayat Aksi
[11] Ekspor Dokumen ke TXT
[12] Simpan & Keluar
=====
Pilih menu: 3

=====
BUKA & EDIT DOKUMEN
=====
ID      Judul      Baris  Dibuat
-----
1      Tugas tekmul      4      2026-02-25 12:11:33
2      Plottingan        0      2026-05-19 16:07:23
3      12                 0      2026-06-26 14:31:38

Masukkan ID dokumen yang ingin dibuka: |
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

[9] Redo Aksi
[10] Lihat Riwayat Aksi
[11] Ekspor Dokumen ke TXT
[12] Simpan & Keluar
=====
Pilih menu: 2

=====
DAFTAR SEMUA DOKUMEN
=====
ID      Judul      Baris  Dibuat
-----
1      Tugas tekmul      4      2026-02-25 12:11:33
2      Plottingan        0      2026-05-19 16:07:23
3      12                 0      2026-06-26 14:31:38

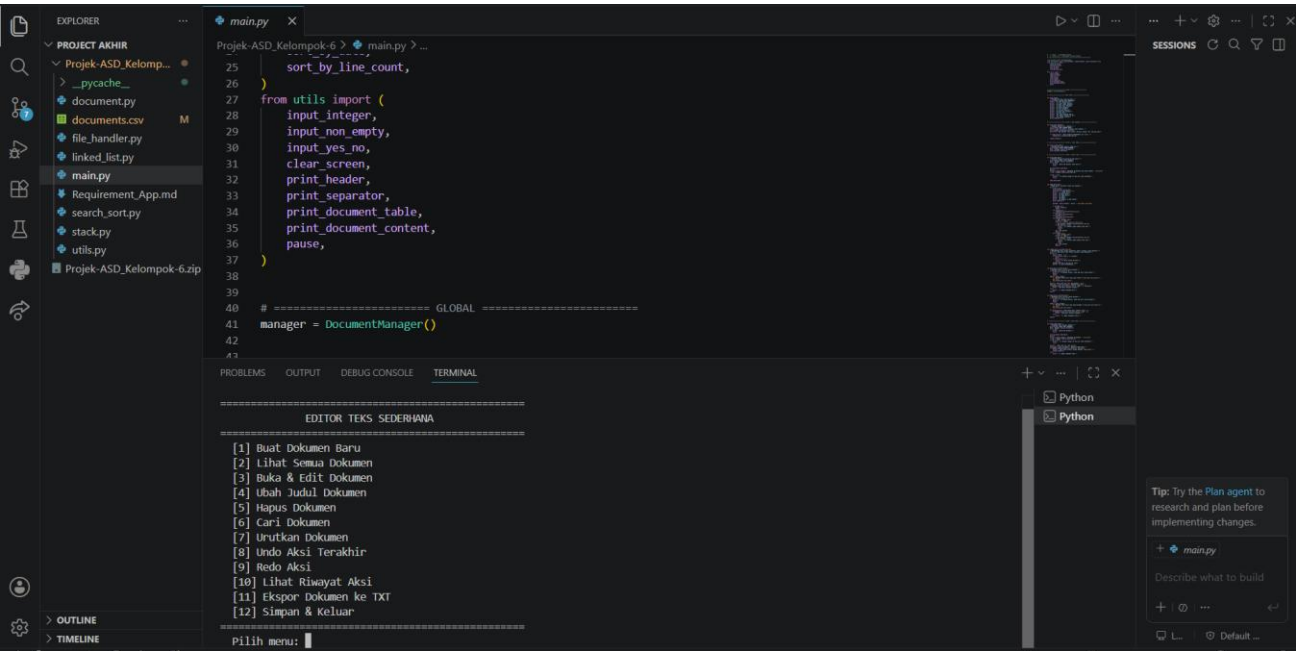
Tekan Enter untuk melanjutkan...|
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

[3] Buka & Edit Dokumen
[4] Ubah Judul Dokumen
[5] Hapus Dokumen
[6] Cari Dokumen
[7] Urutkan Dokumen
[8] Undo Aksi Terakhir
[9] Redo Aksi
[10] Lihat Riwayat Aksi
[11] Ekspor Dokumen ke TXT
[12] Simpan & Keluar
=====
Pilih menu: 1

=====
BUAT DOKUMEN BARU
=====
Masukkan judul dokumen: |
```

Ln 1, Col 1 Spaces: 4 UTF-8 CRLF

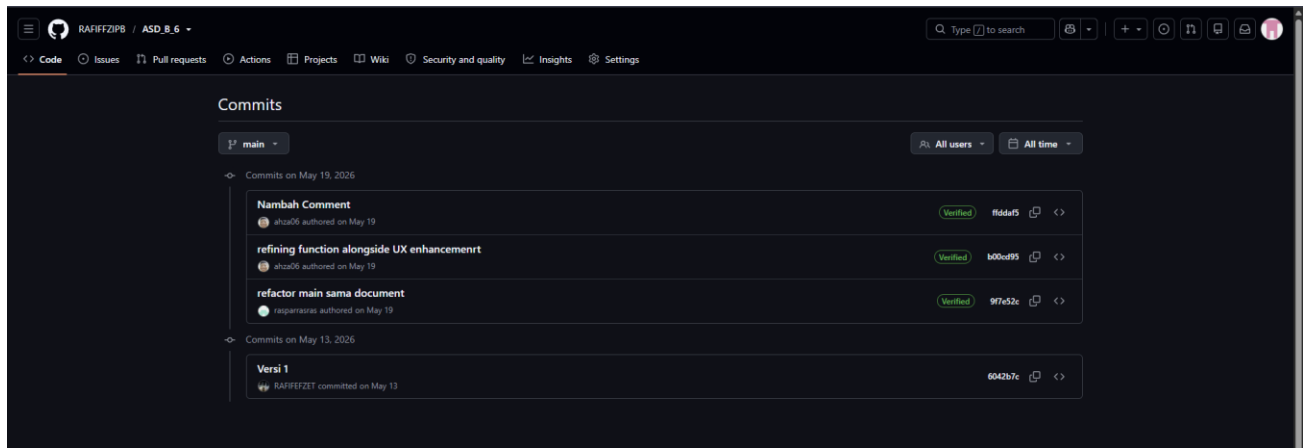


Lampiran C. Struktur Folder Proyek

Name	Date modified	Type	Size
.git	20/05/2026 14.17	File folder	
__pycache__	13/05/2026 14.42	File folder	
document	13/05/2026 13.28	Python Source File	11 KB
documents	26/06/2026 14.41	Microsoft Excel Co...	1 KB
file_handler	13/05/2026 13.28	Python Source File	4 KB
linked_list	13/05/2026 13.28	Python Source File	5 KB
main	13/05/2026 13.28	Python Source File	16 KB
Requirement_App	13/05/2026 13.28	Markdown Source ...	2 KB
search_sort	13/05/2026 13.28	Python Source File	4 KB
stack	13/05/2026 13.28	Python Source File	2 KB
Tugas tekmul	26/06/2026 14.41	Text Document	1 KB
utils	13/05/2026 13.28	Python Source File	4 KB

Lampiran D. Commit History GitHub

https://github.com/RAFIFFZIPB/ASD_B_6/commits/main/



Lampiran E. Source Code Utama



```

452     print_header("SELAMAT DATANG")
453     print(" Aplikasi Editor Teks Sederhana")
454     print(" Kelompok 6 - Project Algoritma")
455     print_separator("=")
456     print()
457
458     # Muat data dari file
459     muat_data_awal()
460     pause()
461
462     while True:
463         clear_screen()
464         menu_utama()
465         pilihan = input_integer(" Pilih menu: ", min_val=1, max_val=12)
466
467         if pilihan == 1:
468             fitur_buat_dokumen()
469         elif pilihan == 2:
470             fitur_lihat_semua()
471         elif pilihan == 3:
472             fitur_buka_edit()
473             continue # Skip pause karena editor punya pause sendiri
474         elif pilihan == 4:
475             fitur_ubah_judul()
476         elif pilihan == 5:
477             fitur_hapus_dokumen()
478         elif pilihan == 6:
479             fitur_cari_dokumen()
480         elif pilihan == 7:
481             fitur_urutkan_dokumen()
482         elif pilihan == 8:
483             fitur_undo()
484         elif pilihan == 9:
485             fitur_redo()
486         elif pilihan == 10:
487             fitur_riwat()

```

```

411 def fitur_ekspor_ekst():
412     print()
421     doc_id = input_integer(" Masukkan ID dokumen yang ingin diekspor: ", min_val=1)
422     doc = manager.read_document(doc_id)
423     if doc is None:
424         print(f" [!] Dokumen dengan ID {doc_id} tidak ditemukan.")
425         return
426
427     success, msg = export_document_to_txt(doc)
428     print(f" {'[OK]'} if success else '![!]' {msg}")
429
430
431 # ===== UTILITAS =====
432
433 def simpan_otomatis():
434     """Menyimpan data secara otomatis ke file CSV."""
435     docs = manager.read_all_documents()
436     save_documents(docs)
437
438
439 def muat_data_awal():
440     """Memuat data dari file CSV saat aplikasi dimulai."""
441     docs, msg = load_documents()
442     print(f" {msg}")
443     for doc in docs:
444         manager.documents.append(doc)
445
446
447 # ===== MAIN LOOP =====
448
449 def main():
450     """Fungsi utama aplikasi."""
451     clear_screen()
452     print_header("SELAMAT DATANG")
453     print(" Aplikasi Editor Teks Sederhana")
454     print(" Kelompok 6 - Project Algoritma")
455     print_separator("=")

```

```

384
385     print(f"  {'No':<5} {'Aksi':<20} {'Detail':<30}")
386     print_separator()
387     for i, action in enumerate(reversed(history), 1):
388         aksi = label_aksi.get(action[0], action[0])
389         if action[0] == "create":
390             detail = f"ID dokumen pada index {action[1]}"
391         elif action[0] == "delete":
392             detail = f"Dokumen '{action[2].get('title', '?')}'"
393         elif action[0] == "update_title":
394             detail = f"'{action[2]}' -> '{action[3]}'"
395         elif action[0] == "add_line":
396             detail = f"Doc ID {action[1]}, baris {action[2] + 1}"
397         elif action[0] == "edit_line":
398             detail = f"Doc ID {action[1]}, baris {action[2] + 1}"
399         elif action[0] == "delete_line":
400             detail = f"Doc ID {action[1]}, '{action[3][:20]}'"
401         else:
402             detail = str(action[1:])
403
404         print(f"  {i:<5} {aksi:<20} {detail:<30}")
405
406     print(f"\n  Total aksi tersimpan: {len(history)}")
407
408
409     # ===== FITUR 11: EKSPOR TXT =====
410
411     def fitur_ekspor_txt():
412         """Mengekspor dokumen ke file TXT."""
413         print_header("EKSPOR DOKUMEN KE TXT")
414         docs = manager.read_all_documents()
415         if not docs:
416             print("  (Belum ada dokumen.)")
417             return
418
419         print_document_table(docs)

```

```

349
350     def fitur_undo():
351         """Membatalkan aksi terakhir."""
352         action, msg = manager.undo()
353         print(f"\n {msg}")
354         if action:
355             simpan_otomatis()
356
357
358     def fitur_redo():
359         """Mengulang aksi yang telah di-undo."""
360         action, msg = manager.redo()
361         print(f"\n {msg}")
362         if action:
363             simpan_otomatis()
364
365
366     # ===== FITUR 10: RIWAYAT =====
367
368     def fitur_riwayat():
369         """Menampilkan riwayat aksi (isi Stack)."""
370         print_header("RIWAYAT AKSI (STACK)")
371         history = manager.get_undo_history()
372         if not history:
373             print("  (Riwayat kosong)")
374             return
375
376         label_aksi = {
377             "create": "Buat dokumen",
378             "delete": "Hapus dokumen",
379             "update_title": "Ubah judul",
380             "add_line": "Tambah baris",
381             "edit_line": "Edit baris",
382             "delete_line": "Hapus baris",
383         }
384

```

```

310         return
311
312     print(" [1] Urutkan berdasarkan Judul (A-Z)")
313     print(" [2] Urutkan berdasarkan Judul (Z-A)")
314     print(" [3] Urutkan berdasarkan Tanggal Dibuat (Terlama)")
315     print(" [4] Urutkan berdasarkan Tanggal Dibuat (Terbaru)")
316     print(" [5] Urutkan berdasarkan Jumlah Baris (Sedikit-Banyak)")
317     print(" [6] Urutkan berdasarkan Jumlah Baris (Banyak-Sedikit)")
318     print(" [0] Kembali")
319     print_separator()
320
321     pilihan = input_integer(" Pilih: ", min_val=0, max_val=6)
322     if pilihan == 0:
323         return
324
325     sorted_docs = []
326     if pilihan == 1:
327         sorted_docs = sort_by_title(docs, ascending=True)
328         print("\n Hasil pengurutan (Judul A-Z):")
329     elif pilihan == 2:
330         sorted_docs = sort_by_title(docs, ascending=False)
331         print("\n Hasil pengurutan (Judul Z-A):")
332     elif pilihan == 3:
333         sorted_docs = sort_by_date(docs, ascending=True)
334         print("\n Hasil pengurutan (Terlama):")
335     elif pilihan == 4:
336         sorted_docs = sort_by_date(docs, ascending=False)
337         print("\n Hasil pengurutan (Terbaru):")
338     elif pilihan == 5:
339         sorted_docs = sort_by_line_count(docs, ascending=True)
340         print("\n Hasil pengurutan (Baris: Sedikit-Banyak):")
341     elif pilihan == 6:
342         sorted_docs = sort_by_line_count(docs, ascending=False)
343         print("\n Hasil pengurutan (Baris: Banyak-Sedikit):")
344
345     print_document_table(sorted_docs)

```

```

267 def fitur_cari_dokumen():
273     print_separator()
274
275     pilihan = input_integer(" Pilih: ", min_val=0, max_val=2)
276     if pilihan == 0:
277         return
278
279     keyword = input_non_empty(" Masukkan kata kunci: ")
280     docs = manager.read_all_documents()
281
282     if pilihan == 1:
283         results = search_by_title(docs, keyword)
284         if results:
285             print(f"\n Ditemukan {len(results)} dokumen:")
286             print_document_table(results)
287         else:
288             print(f"\n Tidak ada dokumen dengan judul mengandung '{keyword}'.")
289
290     elif pilihan == 2:
291         results = search_by_content(docs, keyword)
292         if results:
293             print(f"\n Ditemukan di {len(results)} dokumen:")
294             for doc, line_indices in results:
295                 print(f"\n [{doc.doc_id}] {doc.title}:")
296                 for idx in line_indices:
297                     print(f"    Baris {idx + 1}: {doc.lines[idx]}")
298             else:
299                 print(f"\n Tidak ada konten yang mengandung '{keyword}'.")
300
301
302 # ===== FITUR 7: URUTKAN DOKUMEN =====
303
304 def fitur_urutkan_dokumen():
305     """Fitur pengurutan dokumen (SORTING)."""
306     print_header("URUTKAN DOKUMEN")
307     docs = manager.read_all_documents()
308     if not docs:

```

```

245         print(" (Belum ada dokumen.)")
246         return
247
248     print_document_table(docs)
249     print()
250     doc_id = input_integer(" Masukkan ID dokumen yang ingin dihapus: ", min_val=1)
251     _, doc = manager.read_document(doc_id)
252     if doc is None:
253         print(f" [!] Dokumen dengan ID {doc_id} tidak ditemukan.")
254         return
255
256     print(f" Dokumen: {doc.title} ({doc.get_line_count()} baris)")
257     if input_yes_no(" Yakin ingin menghapus? (y/n): "):
258         if manager.delete_document(doc_id):
259             print(" [OK] Dokumen berhasil dihapus.")
260             simpan_otomatis()
261         else:
262             print(" [!] Gagal menghapus dokumen.")
263
264
265 # ===== FITUR 6: CARI DOKUMEN =====
266
267 def fitur_cari_dokumen():
268     """Fitur pencarian dokumen (SEARCHING)."""
269     print_header("CARI DOKUMEN")
270     print(" [1] Cari berdasarkan Judul")
271     print(" [2] Cari berdasarkan Isi/Konten")
272     print(" [0] Kembali")
273     print_separator()
274
275     pilihan = input_integer(" Pilih: ", min_val=0, max_val=2)
276     if pilihan == 0:
277         return
278
279     keyword = input_non_empty(" Masukkan kata kunci: ")

```

Share

RAFIFFETZET (1 month ago) Ln 8, Col 36 (2 selected) Spaces: 4 UTF-8 CRLF Python

```

205         print(" [OK] Baris berhasil dihapus.")
206     else:
207         print(" [!] Gagal menghapus baris.")
208     pause()
209
210
211 # ===== FITUR 4: UBAH JUDUL =====
212
213 def fitur_ubah_judul():
214     """Mengubah judul dokumen (UPDATE)."""
215     print_header("UBAH JUDUL DOKUMEN")
216     docs = manager.read_all_documents()
217     if not docs:
218         print(" (Belum ada dokumen.)")
219         return
220
221     print_document_table(docs)
222     print()
223     doc_id = input_integer(" Masukkan ID dokumen: ", min_val=1)
224     _, doc = manager.read_document(doc_id)
225     if doc is None:
226         print(f" [!] Dokumen dengan ID {doc_id} tidak ditemukan.")
227         return
228
229     print(f" Judul saat ini: {doc.title}")
230     new_title = input_non_empty(" Masukkan judul baru: ")
231     if manager.update_document_title(doc_id, new_title):
232         print(f" [OK] Judul berhasil diubah menjadi '{new_title}'.")
233         simpan_otomatis()
234     else:
235         print(" [!] Gagal mengubah judul.")
236
237
238 # ===== FITUR 5: HAPUS DOKUMEN =====
239
240 def fitur_hapus_dokumen():

```

Share

RAFIFFETZET (1 month ago) Ln 8, Col 36 (2 selected) Spaces: 4 UTF-8 CRLF Python

```

172 def edit_baris_interaktif(doc):
173     """Mengedit baris tertentu dalam dokumen."""
174     if doc.get_line_count() == 0:
175         print("\n [!] Dokumen kosong. Tidak ada baris untuk diedit.")
176         pause()
177         return
178     nomor = input_integer(
179         f" Masukkan nomor baris yang ingin diedit (1-{doc.get_line_count()}): ",
180         min_val=1,
181         max_val=doc.get_line_count(),
182     )
183     print(f" Baris saat ini: {doc.lines[nomor - 1]}")
184     teks_baru = input_non_empty(" Masukkan teks baru: ")
185     if manager.edit_line_in_doc(doc.doc_id, nomor - 1, teks_baru):
186         print(" [OK] Baris berhasil diubah.")
187     else:
188         print(" [!] Gagal mengubah baris.")
189     pause()
190
191
192 def hapus_baris_interaktif(doc):
193     """Menghapus baris tertentu dalam dokumen."""
194     if doc.get_line_count() == 0:
195         print("\n [!] Dokumen kosong. Tidak ada baris untuk dihapus.")
196         pause()
197         return
198     nomor = input_integer(
199         f" Masukkan nomor baris yang ingin dihapus (1-{doc.get_line_count()}): ",
200         min_val=1,
201         max_val=doc.get_line_count(),
202     )
203     if input_yes_no(f" Yakin hapus baris {nomor}? (y/n): "):
204         if manager.delete_line_in_doc(doc.doc_id, nomor - 1):
205             print(" [OK] Baris berhasil dihapus.")
206         else:
207             print(" [!] Gagal menghapus baris.")
208     pause()

```

```

174     if doc.get_line_count() == 0:
175         print("\n [!] Dokumen kosong. Tidak ada baris untuk diedit.")
176         pause()
177         return
178     nomor = input_integer(
179         f" Masukkan nomor baris yang ingin diedit (1-{doc.get_line_count()}): ",
180         min_val=1,
181         max_val=doc.get_line_count(),
182     )
183     print(f" Baris saat ini: {doc.lines[nomor - 1]}")
184     teks_baru = input_non_empty(" Masukkan teks baru: ")
185     if manager.edit_line_in_doc(doc.doc_id, nomor - 1, teks_baru):
186         print(" [OK] Baris berhasil diubah.")
187     else:
188         print(" [!] Gagal mengubah baris.")
189     pause()
190
191
192 def hapus_baris_interaktif(doc):
193     """Menghapus baris tertentu dalam dokumen."""
194     if doc.get_line_count() == 0:
195         print("\n [!] Dokumen kosong. Tidak ada baris untuk dihapus.")
196         pause()
197         return
198     nomor = input_integer(
199         f" Masukkan nomor baris yang ingin dihapus (1-{doc.get_line_count()}): ",
200         min_val=1,
201         max_val=doc.get_line_count(),
202     )
203     if input_yes_no(f" Yakin hapus baris {nomor}? (y/n): "):
204         if manager.delete_line_in_doc(doc.doc_id, nomor - 1):
205             print(" [OK] Baris berhasil dihapus.")
206         else:
207             print(" [!] Gagal menghapus baris.")
208     pause()

```



```

110 def mode_editor(doc):
111     doc = doc_refresh
112     pause()
113     elif pilihan == 5:
114         _, msg = manager.redo()
115         print(f"\n {msg}")
116         _, doc_refresh = manager.read_document(doc.doc_id)
117         if doc_refresh is None:
118             print(" [!] Dokumen sudah dihapus oleh redo.")
119             pause()
120             break
121         doc = doc_refresh
122         pause()
123
124 def tambah_baris_interaktif(doc_id):
125     """Menambahkan baris secara interaktif. Ketik 'selesai' untuk berhenti."""
126     print("\n Ketik baris teks (ketik 'selesai' untuk berhenti):")
127     while True:
128         text = input(" > ")
129         if text.strip().lower() == "selesai":
130             break
131         if text.strip() == "":
132             print(" [!] Baris kosong dilewati.")
133             continue
134         manager.add_line_to_doc(doc_id, text)
135         print(" [+] Baris ditambahkan.")
136
137 def edit_baris_interaktif(doc):
138     """Mengedit baris tertentu dalam dokumen."""
139     if doc.get_line_count() == 0:
140         print("\n [!] Dokumen kosong. Tidak ada baris untuk diedit.")
141         pause()
142         return
143     nomor = input_integer(
144         f" Masukkan nomor baris yang ingin diedit (1-{doc.get_line_count()}): ",

```

```

109
110 def mode_editor(doc):
111     """Mode editor interaktif untuk satu dokumen."""
112     while True:
113         clear_screen()
114         print_document_content(doc)
115         print("\n --- Menu Editor ---")
116         print(" [1] Tambah Baris")
117         print(" [2] Edit Baris")
118         print(" [3] Hapus Baris")
119         print(" [4] Undo")
120         print(" [5] Redo")
121         print(" [0] Kembali ke Menu Utama")
122         print_separator()
123
124         pilihan = input_integer(" Pilih: ", min_val=0, max_val=5)
125
126         if pilihan == 0:
127             simpan_otomatis()
128             break
129         elif pilihan == 1:
130             tambah_baris_interaktif(doc.doc_id)
131         elif pilihan == 2:
132             edit_baris_interaktif(doc)
133         elif pilihan == 3:
134             hapus_baris_interaktif(doc)
135         elif pilihan == 4:
136             _, msg = manager.undo()
137             print(f"\n {msg}")
138             # Refresh referensi dokumen setelah undo
139             _, doc_refresh = manager.read_document(doc.doc_id)
140             if doc_refresh is None:
141                 print(" [!] Dokumen sudah dihapus oleh undo.")
142                 pause()
143                 break
144             doc = doc_refresh
145             pause()

```

```

77
78
79 # ===== FITUR 2: LIHAT SEMUA =====
80
81 def fitur_lihat_semua():
82     """Menampilkan semua dokumen (READ ALL)."""
83     print_header("DAFTAR SEMUA DOKUMEN")
84     docs = manager.read_all_documents()
85     print_document_table(docs)
86
87
88 # ===== FITUR 3: BUKA & EDIT =====
89
90 def fitur_buka_edit():
91     """Membuka dokumen dan masuk ke mode editor."""
92     print_header("BUKA & EDIT DOKUMEN")
93     docs = manager.read_all_documents()
94     if not docs:
95         print(" (Belum ada dokumen. Buat dulu!)")
96         return
97
98     print_document_table(docs)
99     print()
100     doc_id = input_integer(" Masukkan ID dokumen yang ingin dibuka: ", min_val=1)
101     _, doc = manager.read_document(doc_id)
102
103     if doc is None:
104         print(f" [!] Dokumen dengan ID {doc_id} tidak ditemukan.")
105         return
106
107     mode_editor(doc)
108
109
110 def mode_editor(doc):
111     """Mode editor interaktif untuk satu dokumen."""
112     while True:
113         clear_screen()

```

```

38
39
40 # ===== GLOBAL =====
41 manager = DocumentManager()
42
43
44 # ===== MENU UTAMA =====
45
46 def menu_utama():
47     """Menampilkan menu utama aplikasi."""
48     print_header("EDITOR TEKS SEDERHANA")
49     print(" [1] Buat Dokumen Baru")
50     print(" [2] Lihat Semua Dokumen")
51     print(" [3] Buka & Edit Dokumen")
52     print(" [4] Ubah Judul Dokumen")
53     print(" [5] Hapus Dokumen")
54     print(" [6] Cari Dokumen")
55     print(" [7] Urutkan Dokumen")
56     print(" [8] Undo Aksi Terakhir")
57     print(" [9] Redo Aksi")
58     print(" [10] Lihat Riwayat Aksi")
59     print(" [11] Ekspor Dokumen ke TXT")
60     print(" [12] Simpan & Keluar")
61     print_separator("=")
62
63
64 # ===== FITUR 1: BUAT DOKUMEN =====
65
66 def fitur_buat_dokumen():
67     """Membuat dokumen baru (CREATE)."""
68     print_header("BUAT DOKUMEN BARU")
69     title = input_non_empty(" Masukkan judul dokumen: ")
70     doc = manager.create_document(title)
71     print(f"\n [OK] Dokumen '{doc.title}' berhasil dibuat! (ID: {doc.doc_id})")
72
73     if input_yes_no(" Ingin langsung menambahkan isi? (y/n): ") == "y":
74         tambah_baris_interaktif(doc.doc_id)

```

```

Projek-ASD_Kelompok-6 > main.py > ...
1 # =====
2 # APLIKASI EDITOR TEKS SEDERHANA (CLI)
3 # Kelompok 6 - Project Algoritma & Struktur Data
4 #
5 # Fitur Utama:
6 # - CRUD dokumen lengkap
7 # - Editor teks dengan Undo/Redo (Stack - LIFO)
8 # - Penyimpanan data dokumen (Linked List)
9 # - Sorting & Searching
10 # - File Handling (CSV)
11 # - Validasi input
12 #
13 # Struktur Data yang digunakan:
14 # 1. Stack -> Undo/Redo history
15 # 2. Linked List -> Penyimpanan koleksi dokumen
16 # =====
17
18 from document import DocumentManager
19 from file_handler import save_documents, load_documents, export_document_to_txt
20 from search_sort import (
21     search_by_title,
22     search_by_content,
23     sort_by_title,
24     sort_by_date,
25     sort_by_line_count,
26 )
27 from utils import (
28     input_integer,
29     input_non_empty,
30     input_yes_no,
31     clear_screen,
32     print_header,
33     print_separator,
34     print_document_table,
35     print_document_content,
36     pause,
37 )

```

